

Reviewer Pack — Minimal Order of Geometric Entanglement and Communication Co...

Entry 106606d9ed75 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 21:01:46 UTC

Minimal Order of Geometric Entanglement and Communication Complexity Rank Variance

Entry ID: 106606d9ed75 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-07 21:01:46 UTC

1. Conjecture

Field A (mathematical branch): Quantum Information Theory (Geometric Entanglement) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

For any given n -vertex graph G , the minimal geometric entanglement $E(G)$ of its associated commuting matrix $C(G)$ is linearly correlated with its communication complexity rank variance $\text{Var}(C(G))$, such that $E(G) = \Theta(\text{Var}(C(G)))$

Rationale (proposer's reasoning):

Geometric entanglement measures the non-classical correlations in a quantum system. If this correlation can be linked to the complexity of communication tasks on graphs, it might provide insights into the limits of classical computation. This bridge may expose new structures and relationships between quantum and classical complexity theory.

Taxonomy category: Geometric_Quantum_Info (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 7e88ef4c33b06d65

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all n -vertex d -regular graphs G with $n \leq 40$ and across 30 random seeds, the correlation coefficient between geometric entanglement $E(G)$ and communication complexity rank variance $\text{Var}(C(G))$ is ≥ 0.8 AND the mean of the ratio $E(G)/\text{Var}(C(G))$ is within a factor of 3 from its expected value under the null hypothesis.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	SAFE	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_d_regular_graph(n, d):
        if (d * n) % 2 != 0 or d > n - 1:
            return None
        G = [[0] * n for _ in range(n)]
        edges = set()
        for i in range(d):
            for j in range(i + 1, n):
                if len(edges) == (d * n) // 2:
                    break
                if random.choice([True, False]):
                    G[i][j] = G[j][i] = 1
                    edges.add((i, j))
        return G

    def matrix_multiply(A, B):
        n = len(A)
        C = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(n):
                for k in range(n):
                    C[i][j] += A[i][k] * B[k][j]
        return C

    def gaussian_elimination(A, b):
        n = len(A)
        augmented_matrix = [A[i] + [b[i]] for i in range(n)]
        for i in range(n):
            max_row = i
            for j in range(i + 1, n):
                if abs(augmented_matrix[j][i]) > abs(augmented_matrix[max_row][i]):
```

```

        max_row = j
        augmented_matrix[i], augmented_matrix[max_row] = augmented_matrix[max_row], augmented_matrix[i]
        pivot = augmented_matrix[i][i]
        for j in range(i, n + 1):
            augmented_matrix[i][j] /= pivot
        for j in range(n):
            if j != i:
                factor = augmented_matrix[j][i]
                for k in range(i, n + 1):
                    augmented_matrix[j][k] -= factor * augmented_matrix[i][k]
    return [row[-1] for row in augmented_matrix]

def commuting_matrix(G):
    n = len(G)
    I = [[1 if i == j else 0 for j in range(n)] for i in range(n)]
    A = G
    B = I
    C = matrix_multiply(A, B)
    D = matrix_multiply(B, A)
    return [C[i][j] - D[i][j] for i in range(n) for j in range(n)]

def geometric_entanglement(C):
    n = int(math.sqrt(len(C)))
    eigenvalues = []
    for i in range(n):
        eigenvector = [0] * n
        eigenvector[i] = 1
        value = sum(C[k][i] * eigenvector[k] for k in range(n))
        eigenvalues.append(value)
    return max(eigenvalues) - min(eigenvalues)

def communication_complexity_rank_variance(C):
    n = int(math.sqrt(len(C)))
    rank = 0
    for i in range(n):
        row = C[i * n:(i + 1) * n]
        if any(row[j] != 0 for j in range(n)):
            rank += 1
    return n - rank

def generate_random_d_regular_graphs(n, d, k):
    graphs = []
    for _ in range(k):
        graph = generate_d_regular_graph(n, d)
        if graph is not None:
            graphs.append(graph)
    return graphs

n_values = [5, 10, 15, 20, 30, 40]
instances_tested = 0
total_metric_value = 0
max_n = 0
conjecture_holds = True
counterexample = ""

for n in n_values:

```

```

graphs = generate_random_d_regular_graphs(n, d=2, k=5)
for graph in graphs:
    instances_tested += 1
    max_n = max(max_n, n)
    C = commuting_matrix(graph)
    E_G = geometric_entanglement(C)
    Var_C_G = communication_complexity_rank_variance(C)
    total_metric_value += E_G * Var_C_G

if instances_tested < 30:
    conjecture_holds = False
    counterexample = "insufficient_instances"

mean_metric_value = total_metric_value / instances_tested
support_fraction = Fraction(instances_tested, len(n_values) * 5)

return {
    "metric_name": "E(G) * Var(C(G))",
    "metric_value": mean_metric_value,
    "instances_tested": instances_tested,
    "n_max": max_n,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = Fraction(sum(1 for r in results if r["conjecture_holds"]), len(results))

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")
    elif support_fraction >= Fraction(8, 10):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"insufficient_instances\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c12de902.py", line 138, in <module>

```

    result = run_trial(seed)
    ^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c12de902.py", line 112, in run_trial
    E_G = geometric_entanglement(C)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c12de902.py", line 78, in geometric_ent
    value = sum(C[k][i] * eigenvector[k] for k in range(n))
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c12de902.py", line 78, in <genexpr>
    value = sum(C[k][i] * eigenvector[k] for k in range(n))
    ~~~~^^^
TypeError: 'int' object is not subscriptable

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating the conjecture's support conditions. | next: Investigate and fix the error in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14802
2	preregistration	ollama_remote	glm4:latest	0	0	19887
3	novelty	ollama_remote	glm4:latest	0	0	8877
4	novelty	ollama_remote	glm4:latest	0	0	8547
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	53617
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15655
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16933
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15491
9	judge	ollama_remote	glm4:latest	0	0	16840

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 170648 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/106606d9ed75.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/106606d9ed75.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/106606d9ed75.tar.gz` (if generated)